

Diseño y desarrollo de servicios web - caso. **GA7-220501096-AA5-EV01.**

victor anderson castro fuerte

Servicio Nacional de Aprendizaje SENA Tecnología en análisis y
desarrollo de software

Ficha No 3186613.

Julio

2026

Tomando como referencia lo visto en el componente formativo “Construcción API”, realizar el diseño y la codificación de un servicio web para el siguiente caso:

- Se requiere realizar un servicio web para un registro y un inicio de sesión. El servicio recibirá un usuario y una contraseña, si la autenticación es correcta saldrá un mensaje de autenticación satisfactoria, en caso contrario, debe devolver error en la autenticación.
- El código debe contener comentarios.
- Se debe crear el proyecto utilizando herramientas de versionamiento.

Objetivo

Desarrollar un servicio web que permita:

- Registrar un usuario.
- Iniciar sesión.
- Validar usuario y contraseña.
- Mostrar un mensaje si la autenticación fue correcta.
- Mostrar un error cuando los datos sean incorrectos.

1. Crear el proyecto

Abre Git Bash. Y ejecutar las siguientes

Para cumplir con el requerimiento de herramientas de versionamiento, sigue estos pasos en tu terminal:

```
MINGW64:/c/Users/LENOVO/servicio-auth-caso

LENOVO@DESKTOP-NO3CS7C MINGW64 ~ (master)
$ mkdir servicio-auth-caso

LENOVO@DESKTOP-NO3CS7C MINGW64 ~ (master)
$ cd servicio-auth-caso

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ git init
Initialized empty Git repository in C:/Users/LENOVO/servicio-auth-caso/.git/

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ npm init -y
wrote to C:\Users\LENOVO\servicio-auth-caso\package.json:

{
  "name": "servicio-auth-caso",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ npm install express bcryptjs

added 68 packages, and audited 69 packages in 4s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ echo "node_modules/" > .gitignore

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ git commit -m "Commit inicial: Configuración del proyecto y dependencias"
[master (root-commit) f4e4459] Commit inicial: Configuración del proyecto y dependencias
3 files changed, 890 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 package-lock.json
 create mode 100644 package.json

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ git add server.js

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ git commit -m "Desarrollo: Implementación de endpoints de registro, login y lógica de autenticación"
[master ff9d6a5] Desarrollo: Implementación de endpoints de registro, login y lógica de autenticación
1 file changed, 96 insertions(+)
 create mode 100644 server.js

LENOVO@DESKTOP-NO3CS7C MINGW64 ~/servicio-auth-caso (master)
$ echo "node_modules/" > .gitignore
```

Crea un archivo llamado server.js en la raíz del proyecto. Este código simula una base de datos en memoria

```
server.js  usuarios.js
C:\Users\LENOVO> servicio-auth-caso > .\server.js > app.post("/api/register") callback
1 // =====
2 // Importar dependencias
3 // =====
4 const express = require("express");
5 const bcrypt = require("bcryptjs");
6
7 // Crear la aplicación
8 const app = express();
9
10 // Puerto del servidor
11 const PORT = 3000;
12
13 // Permitir recibir datos en formato JSON
14 app.use(express.json());
15
16 // =====
17 // Base de datos simulada
18 // =====
19 const usuariosDB = [];
20
21 // =====
22 // REGISTRO DE USUARIOS
23 // POST /api/register
24 // =====
25 app.post("/api/register", async (req, res) => {
26
27   try {
28
29     // Obtener los datos enviados desde Postman
30     const { username, password } = req.body;
31
32     // Validar que lleguen los datos
33     if (!username || !password) {
34       return res.status(400).json({
35         error: "El usuario y la contraseña son obligatorios."
36       });
37     }
38
39     // Verificar si el usuario ya existe
40     const usuarioExiste = usuariosDB.find(
41       user => user.username === username
42     );
43
44     if (usuarioExiste) {
45       return res.status(400).json({
46         error: "El usuario ya existe."
47       });
48     }
49
50     // Crear el nuevo usuario
51     const hashedPassword = bcrypt.hashSync(password, 10);
52     const newUser = { username, password: hashedPassword };
53     usuariosDB.push(newUser);
54
55     return res.status(201).json({ message: "Usuario registrado exitosamente." });
56   } catch (error) {
57     console.error(error);
58     return res.status(500).json({ error: "Error interno del servidor." });
59   }
60 });
61
62 // Iniciar el servidor
63 app.listen(PORT, () => {
64   console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
65 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\servicio-login> cd C:\Users\LENOVO\servicio-auth-caso
PS C:\Users\LENOVO\servicio-auth-caso> node server.js
Servidor ejecutándose en http://localhost:3000
PS C:\Users\LENOVO\servicio-auth-caso>
```

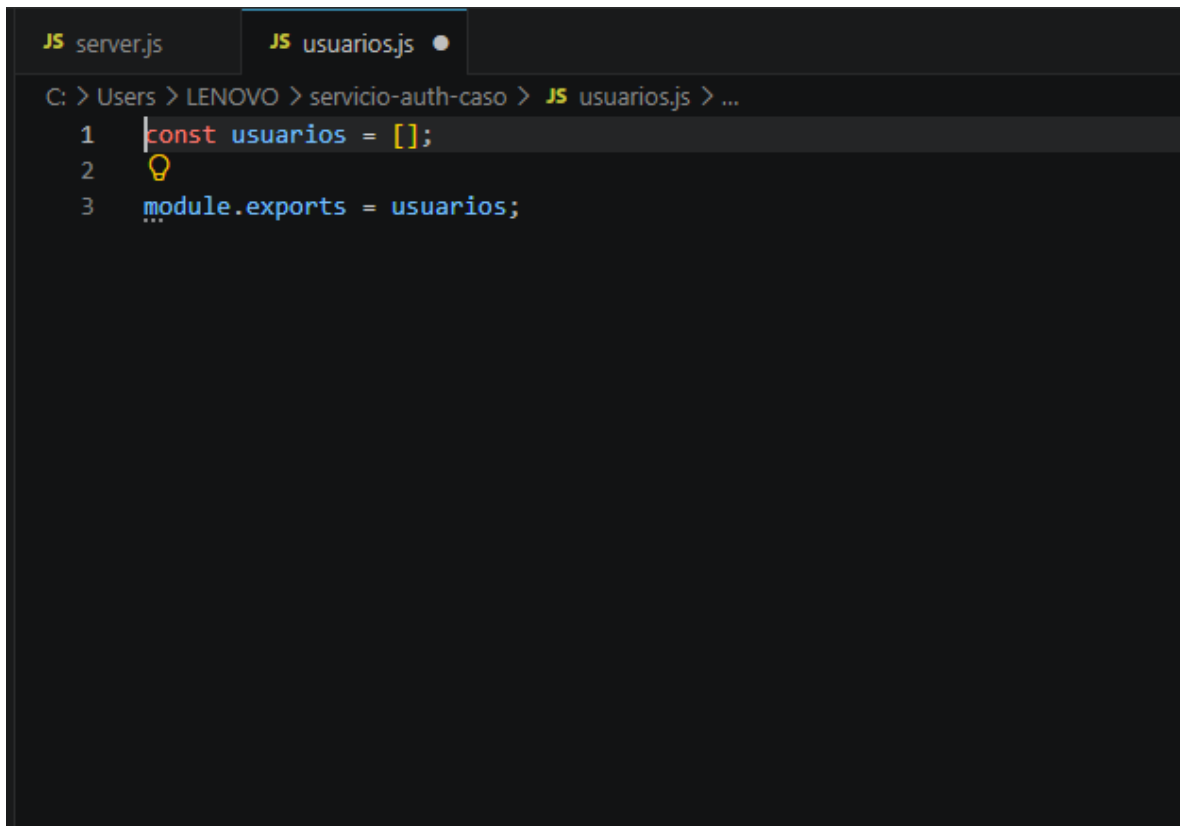
```
JS server.js x JS usuarios.js
C:\> Users > LENOVO > servicio-auth-caso > JS server.js > app.post("/api/register") callback
25 app.post("/api/register", async (req, res) => {
39     // Verificar si el usuario ya existe
40     const usuarioExiste = usuariosDB.find(
41         user => user.username === username
42     );
43
44     if (usuarioExiste) {
45         return res.status(400).json({
46             error: "El usuario ya existe."
47         });
48     }
49
50     // Encriptar la contraseña
51     const salt = await bcrypt.genSalt(10);
52     const passwordEncriptada = await bcrypt.hash(password, salt);
53
54     // Guardar usuario
55     usuariosDB.push({
56         username: username,
57         password: passwordEncriptada
58     });
59
60     // Respuesta exitosa
61     res.status(201).json({
62         mensaje: "Usuario registrado correctamente."
63     });
64
65 } catch (error) {
66
67     // Mostrar el error en la terminal
68     console.error(error);
69
70     res.status(500).json({
71         error: "Error interno del servidor al registrar."
72     });
73
74 }
75
76 });
77
78 // =====
79 // LOGIN
80 // POST /api/login
81 // =====
82 app.post("/api/login", async (req, res) => {
83
84     try {
85         const { username, password } = req.body;
```

```
JS server.js X JS usuarios.js
C:\Users\LENOVO > servicio-auth-caso > JS server.js > app.post("/api/register") callback
82 app.post("/api/login", async (req, res) => {
86   const { username, password } = req.body;
87
88   // Validar datos
89   if (!username || !password) {
90
91     return res.status(400).json({
92       error: "Debe ingresar usuario y contraseña."
93     });
94   }
95
96   // Buscar usuario
97   const usuario = usuariosDB.find(
98     user => user.username === username
99   );
100
101   // Validar existencia
102   if (!usuario) {
103
104     return res.status(401).json({
105       error: "Error en la autenticación."
106     });
107   }
108
109   // Comparar contraseña
110   const passwordCorrecta = await bcrypt.compare(
111     password,
112     usuario.password
113   );
114
115   if (passwordCorrecta) {
116
117     return res.status(200).json({
118       mensaje: "Autenticación satisfactoria."
119     });
120   }
121
122   return res.status(401).json({
123     error: "Error en la autenticación."
124   });
125
126 } catch (error) {
127
128   console.error(error);
129
130   return res.status(500).json({
131
Ln 25, Col 48
```

```

JS server.js  X  JS usuarios.js
C: > Users > LENOVO > servicio-auth-caso > JS server.js > app.post("/api/register") callback
82  app.post("/api/login", async (req, res) => {
112      const passwordCorrecta = await bcrypt.compare(
113          password,
114          usuario.password
115      );
116
117      if (passwordCorrecta) {
118          return res.status(200).json({
119              mensaje: "Autenticación satisfactoria."
120          });
121      }
122
123      return res.status(401).json({
124          error: "Error en la autenticación."
125      });
126
127      } catch (error) {
128
129          console.error(error);
130
131          return res.status(500).json({
132              error: "Error interno del servidor al autenticar."
133          });
134      }
135  });
136
137  // =====
138  // Ruta principal (opcional)
139  // =====
140  app.get("/", (req, res) => {
141
142      res.send("API funcionando correctamente");
143  });
144
145  // =====
146  // Iniciar servidor
147  // =====
148  app.listen(PORT, () => {
149
150      console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
151  });
152
153
154
155
156
157

```



```
JS server.js JS usuarios.js ●
C: > Users > LENOVO > servicio-auth-caso > JS usuarios.js > ...
1  const usuarios = [];
2
3  module.exports = usuarios;
```

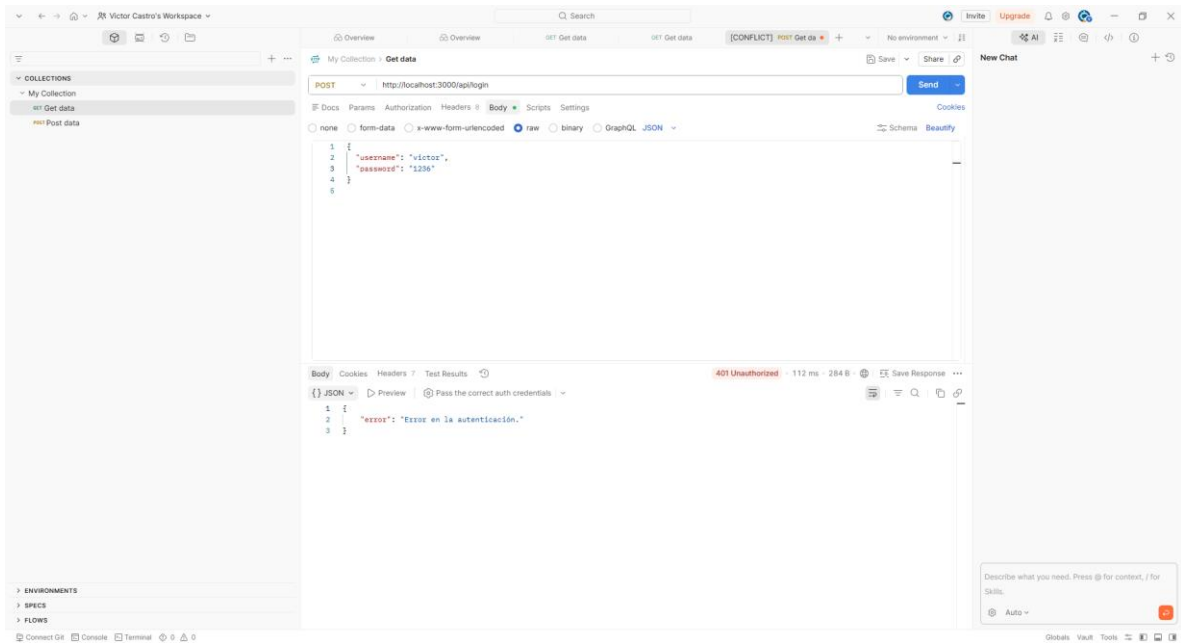
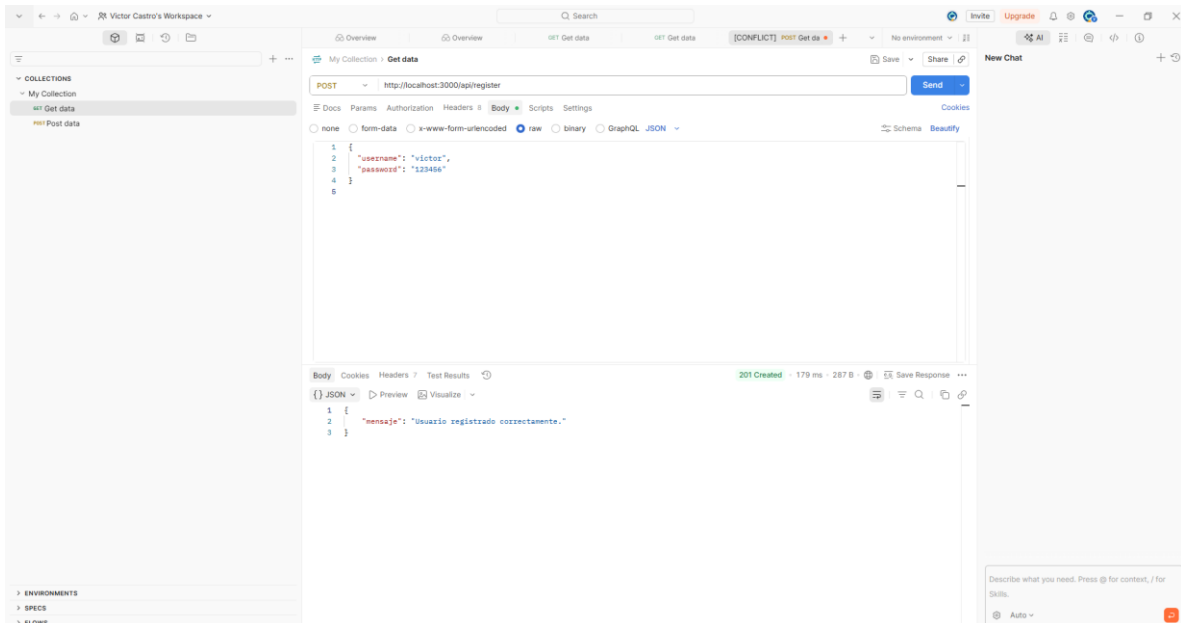
Guardar Cambios en el Repositorio

Una vez finalizado el código, guarda el archivo y registra el avance en Git ejecutando en tu terminal:

Pruebas de Funcionamiento (Evidencia)

Para probar este servicio web, puedes ejecutar el servidor con el comando `node server.js` y usar herramientas como **Postman**

Y hacemos las pruebas en postman creado primero un usuario y después nos dirigimos a `http://localhost:3000/api/login` para verificar que quedo guardado y colocamos mal la contraseña para verificar



Acá agrego el link de repositorio <https://github.com/vc845557-alt/repositorio>